

README

SM4 对称加密算法实验 —— CBC 模式加解密实现 (C++)

一、实验目的

- 理解国密算法 **SM4** 的结构与工作原理；
- 熟悉对称分组密码的加密模式（如 ECB、CBC）；
- 通过编程实现 **SM4** 算法在 **CBC** 模式下的加密与解密；
- 掌握分组填充（PKCS#7）与 IV 初始化向量的使用。

二、SM4 算法简介

SM4 是中国商用密码标准中定义的对称加密算法，具有以下特性：

- 块长 (Block size) : 128 位 (16 字节)；
- 密钥长度 (Key size) : 128 位 (16 字节)；
- 轮数 (Rounds) : 32 轮；
- 算法结构：非 Feistel 型结构，使用 S 盒、线性变换等。

本实验中使用 **CBC 模式 (Cipher Block Chaining)**，即每个明文块与上一个密文块进行异或后再加密，第一个块使用 IV（初始向量）。

三、实验环境

- 编程语言：C++
- 编译器：支持 C++11 及以上的编译器（如 g++、MSVC）
- 标准库：<vector>, <cstring>, <iostream>

四、CBC 模式工作流程

对于明文分组 $P_1, P_2, \dots, P_n$ ，密钥 K ，初始向量 IV ：

加密流程：

- $C_0 = IV$
- 对于 $i = 1$ 到 n ：
 - $C_i = \text{Enc}_K(P_i \oplus C_{i-1})$

解密流程:

1. 对于 $i = 1$ 到 n :
 - $P_i = \text{Dec}_K(C_i) \oplus C_{i-1}$

其中 Enc_K 和 Dec_K 分别为 SM4 的加密和解密操作。

五、实验关键实现说明

1. 加密函数 `EncryptCBC`

- 使用 16 字节对齐的填充 (PKCS#7 padding) ;
- 每个块加密前先与 IV 或上一个密文块进行 XOR;
- 使用 SM4 核心轮函数 `OneRound` 完成加密。

2. 解密函数 `DecryptCBC`

- 每轮解密结果需与上一个密文块异或;
- 解密后移除填充, 验证 padding 值是否合法;
- 若格式非法 (padding 无效) 返回解密失败。

3. SM4 核心部分

- 包括 `SetKey()`、`OneRound()`、`RoundF()`、S 盒替换和线性变换;
- 轮密钥通过密钥扩展生成共 32 个;
- 算法使用查表的 Sbox 和固定轮常数 CK。

详细代码:

```
class SM4 {
public:
    static void EncryptCBC(const uint8_t* src, size_t len,
        std::vector<uint8_t>& dst,
        uint8_t iv[16],
        const uint8_t key[16]) {
        uint32_t sk[32];
        SetKey(sk, key);

        size_t padding = 16 - (len % 16);
        size_t paddedLen = len + padding;
        dst.resize(paddedLen);

        std::vector<uint8_t> buf(paddedLen);
        memcpy(buf.data(), src, len);
        memset(buf.data() + len, static_cast<int>(padding), padding);

        for (size_t i = 0; i < paddedLen; i += 16) {
            for (size_t j = 0; j < 16; ++j)
                buf[i + j] ^= iv[j];
            OneRound(sk, buf.data() + i, dst.data() + i);
            memcpy(iv, dst.data() + i, 16);
        }
    }
};
```

```

    }
}

static bool DecryptCBC(const uint8_t* src, size_t len,
    std::vector<uint8_t>& dst,
    uint8_t iv[16],
    const uint8_t key[16]) {
    if (len == 0 || len % 16 != 0) return false;

    uint32_t sk[32];
    SetKey(sk, key);
    for (size_t i = 0; i < 16; ++i)
        SWAP(sk[i], sk[31 - i]);

    dst.resize(len);
    std::vector<uint8_t> block(16);
    for (size_t i = 0; i < len; i += 16) {
        memcpy(block.data(), src + i, 16);
        OneRound(sk, src + i, dst.data() + i);
        for (size_t j = 0; j < 16; ++j)
            dst[i + j] ^= iv[j];
        memcpy(iv, block.data(), 16);
    }

    uint8_t padding = dst[len - 1];
    if (padding < 1 || padding > 16) return false;
    for (size_t i = len - padding; i < len; ++i) {
        if (dst[i] != padding) return false;
    }
    dst.resize(len - padding);
    return true;
}

```

private:

```

static void SetKey(uint32_t SK[32], const uint8_t key[16]) {
    uint32_t MK[4], k[36];
    GET_ULONG_BE(MK[0], key, 0);
    GET_ULONG_BE(MK[1], key, 4);
    GET_ULONG_BE(MK[2], key, 8);
    GET_ULONG_BE(MK[3], key, 12);
    for (int i = 0; i < 4; ++i)
        k[i] = MK[i] ^ FK[i];
    for (int i = 0; i < 32; ++i) {
        k[i + 4] = k[i] ^ CalcCRK(k[i + 1] ^ k[i + 2] ^ k[i + 3] ^ CK[i]);
        SK[i] = k[i + 4];
    }
}

```

```

static void OneRound(const uint32_t sk[32], const uint8_t input[16], uint8_t
output[16]) {
    uint32_t x[36] = { 0 };
    GET_ULONG_BE(x[0], input, 0);
    GET_ULONG_BE(x[1], input, 4);
    GET_ULONG_BE(x[2], input, 8);
    GET_ULONG_BE(x[3], input, 12);
    for (int i = 0; i < 32; ++i)

```

```

        x[i + 4] = RoundF(x[i], x[i + 1], x[i + 2], x[i + 3], sk[i]);
        PUT_ULONG_BE(x[35], output, 0);
        PUT_ULONG_BE(x[34], output, 4);
        PUT_ULONG_BE(x[33], output, 8);
        PUT_ULONG_BE(x[32], output, 12);
    }

    static uint32_t RoundF(uint32_t x0, uint32_t x1, uint32_t x2, uint32_t x3,
uint32_t rk) {
        return x0 ^ Linear(x1 ^ x2 ^ x3 ^ rk);
    }

    static uint32_t CalcRK(uint32_t ka) {
        uint8_t a[4], b[4];
        uint32_t bb = 0;
        PUT_ULONG_BE(ka, a, 0);
        for (int i = 0; i < 4; ++i) b[i] = SBox(a[i]);
        GET_ULONG_BE(bb, b, 0);
        return bb ^ ROTL(bb, 13) ^ ROTL(bb, 23);
    }

    static uint32_t Linear(uint32_t ka) {
        uint8_t a[4], b[4];
        uint32_t bb = 0;
        PUT_ULONG_BE(ka, a, 0);
        for (int i = 0; i < 4; ++i) b[i] = SBox(a[i]);
        GET_ULONG_BE(bb, b, 0);
        return bb ^ ROTL(bb, 2) ^ ROTL(bb, 10) ^ ROTL(bb, 18) ^ ROTL(bb, 24);
    }

    static uint8_t SBox(uint8_t inch) {
        return SboxTable[inch >> 4][inch & 0x0F];
    }

    static const uint8_t SboxTable[16][16];
    static const uint32_t FK[4];
    static const uint32_t CK[32];
};

// 静态常量定义
const uint8_t SM4::SboxTable[16][16] = {

{0xd6,0x90,0xe9,0xfe,0xcc,0xe1,0x3d,0xb7,0x16,0xb6,0x14,0xc2,0x28,0xfb,0x2c,0x05}
,

{0x2b,0x67,0x9a,0x76,0x2a,0xbe,0x04,0xc3,0xaa,0x44,0x13,0x26,0x49,0x86,0x06,0x99}
,

{0x9c,0x42,0x50,0xf4,0x91,0xef,0x98,0x7a,0x33,0x54,0x0b,0x43,0xed,0xcf,0xac,0x62}
,

{0xe4,0xb3,0x1c,0xa9,0xc9,0x08,0xe8,0x95,0x80,0xdf,0x94,0xfa,0x75,0x8f,0x3f,0xa6}
,

{0x47,0x07,0xa7,0xfc,0xf3,0x73,0x17,0xba,0x83,0x59,0x3c,0x19,0xe6,0x85,0x4f,0xa8}
,

```

```

{0x68,0x6b,0x81,0xb2,0x71,0x64,0xda,0x8b,0xf8,0xeb,0x0f,0x4b,0x70,0x56,0x9d,0x35}
,

{0x1e,0x24,0x0e,0x5e,0x63,0x58,0xd1,0xa2,0x25,0x22,0x7c,0x3b,0x01,0x21,0x78,0x87}
,

{0xd4,0x00,0x46,0x57,0x9f,0xd3,0x27,0x52,0x4c,0x36,0x02,0xe7,0xa0,0xc4,0xc8,0x9e}
,

{0xea,0xbf,0x8a,0xd2,0x40,0xc7,0x38,0xb5,0xa3,0xf7,0xf2,0xce,0xf9,0x61,0x15,0xa1}
,

{0xe0,0xae,0x5d,0xa4,0x9b,0x34,0x1a,0x55,0xad,0x93,0x32,0x30,0xf5,0x8c,0xb1,0xe3}
,

{0x1d,0xf6,0xe2,0x2e,0x82,0x66,0xca,0x60,0xc0,0x29,0x23,0xab,0x0d,0x53,0x4e,0x6f}
,

{0xd5,0xdb,0x37,0x45,0xde,0xfd,0x8e,0x2f,0x03,0xff,0x6a,0x72,0x6d,0x6c,0x5b,0x51}
,

{0x8d,0x1b,0xaf,0x92,0xbb,0xdd,0xbc,0x7f,0x11,0xd9,0x5c,0x41,0x1f,0x10,0x5a,0xd8}
,

{0x0a,0xc1,0x31,0x88,0xa5,0xcd,0x7b,0xbd,0x2d,0x74,0xd0,0x12,0xb8,0xe5,0xb4,0xb0}
,

{0x89,0x69,0x97,0x4a,0x0c,0x96,0x77,0x7e,0x65,0xb9,0xf1,0x09,0xc5,0x6e,0xc6,0x84}
,

{0x18,0xf0,0x7d,0xec,0x3a,0xdc,0x4d,0x20,0x79,0xee,0x5f,0x3e,0xd7,0xcb,0x39,0x48}
};

const uint32_t SM4::FK[4] = { 0xa3b1bac6, 0x56aa3350, 0x677d9197, 0xb27022dc };
const uint32_t SM4::CK[32] = {

    0x00070e15,0x1c232a31,0x383f464d,0x545b6269,0x70777e85,0x8c939aa1,0xa8afb6bd,0xc
4cbd2d9,

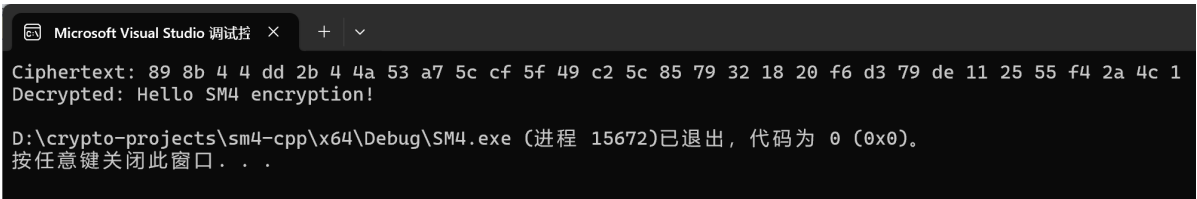
    0xe0e7eef5,0xfc030a11,0x181f262d,0x343b4249,0x50575e65,0x6c737a81,0x888f969d,0xa
4abb2b9,

    0xc0c7ced5,0xdce3eaf1,0xf8ff060d,0x141b2229,0x30373e45,0x4c535a61,0x686f767d,0x8
48b9299,

    0xa0a7aeb5,0xbcc3cad1,0xd8dfe6ed,0xf4fb0209,0x10171e25,0x2c333a41,0x484f565d,0x6
46b7279
};

```

六、输入与输出样例



测试消息：

"Hello SM4 encryption!"

密钥：

01 23 45 67 89 ab cd ef fe dc ba 98 76 54 32 10

控制台输出示例：

Ciphertext: 53 3c e6 0d b5 79 3b c7 1e b2 33 d3 ... (十六进制密文)
Decrypted: Hello SM4 encryption!

七、安全性与正确性说明

- 安全性来源：
 - SM4 算法设计符合国家密码局标准；
 - CBC 模式通过链式依赖增强抗分析能力；
 - 初始向量 IV 每次应重新生成以避免重放攻击。
- 正确性验证：
 - 解密后与原始明文完全一致；
 - 若 padding 或密文格式错误，程序返回失败提示。

八、总结与拓展

- 本实验掌握了国密 SM4 算法的基本结构及加解密流程；
- 熟悉了 CBC 模式的数据依赖与实现方式。

九、附：主函数入口示意

```
int main() {
    uint8_t key[16] = { 0x01, 0x23, ..., 0x10 };
    uint8_t iv[16] = { 0 };
    const char* message = "Hello SM4 encryption!";
    size_t len = strlen(message);

    std::vector<uint8_t> ciphertext;
```

```
SM4::EncryptCBC(reinterpret_cast<const uint8_t*>(message), len, ciphertext,
iv, key);

std::cout << "Ciphertext: ";
for (auto c : ciphertext) std::cout << std::hex << (int)c << " ";

uint8_t iv2[16] = { 0 };
std::vector<uint8_t> plaintext;
if (SM4::DecryptCBC(ciphertext.data(), ciphertext.size(), plaintext, iv2,
key)) {
    std::cout << "Decrypted: ";
    for (auto c : plaintext) std::cout << (char)c;
} else {
    std::cout << "Decrypt failed";
}
}
```